

Dokumentacja - wstępne wyniki

- Mikołaj Garbowski
- Mariusz Pakulski
- Paweł Łasica

Definicja problemu

Badane zagadnienie to agent oparty na wielkim modelu językowym przeznaczony do konwersacyjnej analizy danych ustrukturyzowanych. Kluczowym komponentem takiego systemu jest zadanie text-to-SQL, czyli tłumaczenie zapytania użytkownika w języku naturalnym na zapytanie SQL. Agent ma możliwość wykonywania zapytań SQL w samej bazie danych, a także mechanizmy dostępu do informacji na temat analizowanej bazy danych (np. schemat, przykładowe dane).

W ramach projektu porównamy skuteczność różnych strategii agenta oraz różnych LLM na benchmarku Spider [1]

Studia literaturowe

Zagadnienie wykorzystania modeli językowych do zastosowań z zakresu analizy danych i zadań BI dla baz relacyjnych jest szeroko badane. Typowe rozwiązania obejmują podejścia typu text-to-sql, a także text-to-python i text-to-dsl jako popularne alternatywy do obrabiania ustrukturyzowanych danych [2].

W celu zrozumienia struktury tabeli, typowo dane tabelaryczne są serializowane do jakiejś postaci tekstowej (markdown, json, itp.) co pozwala na bezpośrednie umieszczenie danych w prompcie, natomiast wyzwaniem jest ograniczony rozmiar kontekstu. Badane są także alternatywne podejścia, np. przetwarzanie obrazu tabeli modelami typu VLM, przetwarzanie tabeli w postaci struktur grafowych, a także szczególnie interesujące: traktowanie danych tablicywnych jako odrębnej modalności. Przykładem takiego rozwiązania jest TableGPT2 [3], które łączy koder danych tabelarycznych, produkujący semantyczne osadzenia dla kolumn tabeli, połączony z modelem językowym typu tylko-dekoder wykorzystującym te osadzenia oraz prompt użytkownika.

Sposobem na rozwiązanie problemu z rozmiarem kontekstu, wzbogacenie promptu użytkownika o wiedzę domenową, a także wydobywanie informacji o schemacie bazy danych, stosowane są metody znane z zastosowań typu RAG, zadania *retrieval*.

Zbiory danych

Popularny zbiór WikiSQL [4] obejmuje proste zapytania SQL dotyczące pojedynczych tabel.

Bardziej złożone zapytania SQL (obejmujące złączenia, agregacje itp.) zawierają, znacznie większe zbiory, Spider [1] i bardziej obszerny BIRD [5]. rny Spider [6].

Miary jakości

W literaturze stosowane są następujące miary jakości do oceny rozwiązań typu text-to-SQL:

- Execution accuracy [6] - Określa zgodność wyników zapytania wygenerowanego przez model z wynikami zapytania referencyjnego. Zapytanie uznaje się za poprawne, jeśli po wykonaniu na bazie danych zwraca taki sam rezultat jak zapytanie wzorcowe.
- Exact set match accuracy [6] - Polega na porównaniu struktury zapytania SQL wygenerowanego przez model z zapytaniem referencyjnym. Zapytanie uznaje się za poprawne, jeżeli wszystkie jego komponenty, takie jak SELECT, WHERE, GROUP BY czy ORDER BY, odpowiadają komponentom zapytania wzorcowego.
- Execution Accuracy per SQL Hardness [1] - Dzieli zapytania SQL według poziomu złożoności na cztery kategorie:
 - easy,
 - medium,
 - hard,
 - extra hard.

Klasyfikacja ta opiera się na analizie struktury zapytania SQL, w szczególności na obecności:

- złączeń tabel (JOIN),
 - grupowania danych (GROUP BY),
 - zapytań zagnieżdżonych,
 - operacji zbiorowych,
 - liczby warunków i agregacji.
- Component matching [1], [7] - Umożliwia analizę poprawności poszczególnych komponentów zapytania SQL takich jak:
 - SELECT,
 - WHERE,
 - GROUP BY,
 - ORDER BY,
 - JOIN,
 - nested SQL.

Strategie agentów

Istnieje wiele strategii działania agenta, które różnią się sposobem generowania zapytań SQL, interakcją z bazą danych oraz mechanizmami weryfikacji i poprawy błędów. Niektóre z nich to:

- Plan and Solve [8] - polega na rozdzieleniu procesu rozwiązywania problemu na dwa etapy - utworzenie planu rozwiązania i wykonanie zaplanowanych

kroków. Model językowy najpierw generuje strukturę działania opisującą sposób rozwiązania zadania, a następnie realizuje kolejne kroki prowadzące do wygenerowania zapytania SQL.

- Chain-of-thought [9] - polega na generowaniu rozwiązania poprzez sekwencję kroków rozumowania, bez bezpośredniej interakcji z narzędziami lub środowiskiem wykonawczym w trakcie procesu wnioskowania. Model językowy analizuje zapytanie użytkownika i generuje końcowe zapytanie SQL w jednym przebiegu.
- ReAct - Strategia ReAct (Reason and Act) [9] łączy proces rozumowania z wykonywaniem akcji w środowisku zewnętrznym. Model językowy generuje kolejne kroki działania, wykonuje zapytania SQL lub operacje pomocnicze, analizuje wyniki ich wykonania oraz w razie potrzeby modyfikuje swoje decyzje. W implementacji systemu wykorzystany zostanie komponent SQL Agent frameworka LangChain [10], który domyślnie realizuje logikę działania zgodną z podejściem ReAct. Strategia ta stanowi zatem naturalny wariant bazowy dla systemów konwersacyjnych operujących na bazach danych.

Strategie tworzenia promptów self-correction

- Gentle self-correction prompt [7] - nie zakłada błędu, lecz prosi model o sprawdzenie zapytania i ewentualne wskazanie problemów, podając wskazówki dotyczące elementów SQL do weryfikacji.
- Generic prompt [7] - Zakłada, że zapytanie jest błędne i poleca modelowi zidentyfikować oraz poprawić błędy.

Obie strategie zostały zaimplementowane w strategii zero-shot, czyli bez dodatkowego trenowania modelu na danych specyficznych dla zadania.

Opis rozwiązania

Projekt obejmuje implementację agentów wykorzystujących różne strategie oraz porównanie ich skuteczności.

Zbiór danych

Do ewaluacji jakości wykorzystamy zbiór Spider. WikiSQL uznajemy za nieodpowiedni dla naszego projektu, skoncentrowanego na bardziej złożonych zapytaniach analitycznych. BIRD byłby również dobrym wyborem, natomiast decydujemy się na Spider: również bardzo popularny i mniejszy, co ułatwi nam pracę na ograniczonych zasobach sprzętowych.

Baza danych

Integrujemy agenta z SQLite: uruchamianą lokalnie, relacyjną bazą danych. Jest to typowy wybór przy tego typu zadaniach, używana w popularnych zbiorach

danych [1], [4], [5].

Plan eksperymentów

Planujemy porównać skuteczność opisanych powyżej strategii Plan and Solve, Chain-of-thought oraz ReAct. Planujemy również porównać skuteczność modeli o otwartych wagach dostępne na HuggingFace, wstępnie model z rodziny Llama [11] oraz Bielika [12].

Implementacja

Platforma

Za platformę roboczą przyjmujemy środowisko uruchomieniowe Google Colab. Dostarcza ona możliwość korzystania z dedykowanych kart TPU/GPU oraz uruchamiania notatników Jupyter. W przypadku chęci rozwoju projektu o GUI dopuszczamy możliwość skonteneryzowania aplikacji i uruchamiania jej lokalnie.

Narzędzia

- langchain/langgraph [10], [13] - biblioteki dostarczające rozwiązania dotyczące budowania promptów, agentów oraz zdolności rozumowania
- transformers - biblioteka ułatwiająca pracę z wielkimi modelami językowymi
- pytorch - biblioteka ogólnego przeznaczenia do działań głębokiego uczenia oraz tensorów

Przepływ działania agenta

Krok	Komponent	Rola
1	<code>list_tables</code>	Pobranie listy tabel dostępnych w bazie danych.
2	<code>select_tables</code>	Wybór tabel istotnych dla pytania użytkownika.
3	<code>get_schema</code>	Pobranie schematu wybranych tabel.
4	<code>generate_query</code>	Wygenerowanie zapytania SQLite <code>SELECT</code> .
5	<code>execute_query</code>	Sprawdzenie bezpieczeństwa i wykonanie zapytania.
6	<code>use_all_tables</code>	Awaryjne użycie pełnego schematu, jeżeli wcześniejszy wybór tabel był niewystarczający.
7	<code>correct_query</code>	Korekta zapytania na podstawie błędu.
8	<code>generate_answer</code>	Wygenerowanie krótkiej odpowiedzi dla użytkownika.

W trybie ewaluacyjnym agent może kończyć działanie po wygenerowaniu i ewentualnej korekcie SQL. Jest to potrzebne, ponieważ metryki Spider oceniają zapytania oraz wyniki ich wykonania, a nie opisową odpowiedź w języku naturalnym.

Wariant Chain-of-Thought

Wariant Chain-of-Thought dodaje do generowania SQL krótki plan zapisany w bloku `<think>...</think>`. Plan ma zawierać maksymalnie kilka krótkich informacji:

- istotne tabele,
- potrzebne złączenia,
- filtry, grupowanie, sortowanie lub agregacje,
- kolumny zwracane w wyniku.

Po planie model zwraca jedno zapytanie SQL **SELECT**. Implementacja wyodrębnia plan do osobnego pola diagnostycznego, usuwa go z odpowiedzi modelu i zapisuje samo zapytanie SQL do ewaluacji. Dzięki temu można analizować sposób planowania modelu, ale format predykcji pozostaje zgodny z wymaganiami benchmarku.

Tryb generowania może być wybierany z poziomu argumentów uruchomieniowych, na przykład jako tryb bazowy albo `cot`.

Algorytm self-correction

Self-correction jest uruchamiany, gdy zapytanie nie wykona się poprawnie albo zostanie odrzucone przez walidator bezpieczeństwa. Dla błędów typu `no such column` lub `no such table` agent może najpierw przełączyć się na pełny schemat bazy, ponieważ pierwotnie wybrane tabele mogły być niewystarczające.

Następnie model otrzymuje:

- pytanie użytkownika,
- schemat bazy danych,
- poprzednie zapytanie SQL,
- komunikat błędu.

Na tej podstawie generuje poprawione zapytanie. Domyślnie liczba prób korekty jest ograniczona, na przykład do dwóch. Pozwala to porównać wariant bez korekty, wariant z jedną próbą oraz wariant z większą liczbą prób.

Struktura kodu źródłowego

- **agent** - pakiet z implementacją agenta
 - **agent** - budowanie agenta
 - **nodes** - implementacja węzłów grafu agenta
 - **state** - stan agenta przekazywany między węzłami
 - **utils** - funkcje pomocnicze
 - **make_predictions** - skrypt do generowania predykcji dla zbioru Spider
- **eval** - moduł do ewaluacji wygenerowanych predykcji
 - kod zapożyczony z repozytorium <https://github.com/taoyds/spider>

Instrukcja obsługi

- Do zarządzania projektem Python, zależnościami: `uv`
- Do wykonywania komend: `just`
- Wszystkie komendy opisane w `Justfile`
 - lista wszystkich dostępnych `just -l`
 - opis skryptów `just <komenda> --help`
 - komendy uruchamiane np. `just test`
- Instalacja zależności: `just install`
- Pobranie zbiorów danych `just datasets`
- Przeprowadzenie ewaluacji
 - `just make_predictions` - generuje za pomocą agenta zapytania SQL dla zapytań w języku naturalnym i baz danych ze zbioru Spider.
 - `just eval` - oblicza miary jakości porównując wygenerowane i wzorcowe zapytania SQL

Testy

Wykorzystujemy zaadaptowany skrypt do ewaluacji na zbiorze Spider z repozytorium <https://github.com/taoyds/spider>.

Wstępne wyniki

Poniżej wstępne wyniki dla agenta wykorzystującego model `Qwen/Qwen2.5-1.5B-Instruct`

Eksperyment	Execution accuracy	Exact match
Wariant bazowy	0.137	0.087
Wariant bazowy z self-correction	0.134	0.068
Wariant Chain-of-Thought	0.159	0.117

Wnioski

Zostaną opisane w kolejnych iteracjach projektu.

Bibliografia

- [1] T. Yu *i in.*, „Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. 2018. Dostępne na: <https://arxiv.org/abs/1809.08887>
- [2] J. Tian *i in.*, „Toward Real-World Table Agents: Capabilities, Workflows, and Design Principles for LLM-based Table Intelligence”. 2025. Dostępne na: <https://arxiv.org/pdf/2507.10281>
- [3] A. Su *i in.*, „TableGPT2: A Large Multimodal Model with Tabular Data Integration”. 2024. Dostępne na: <https://arxiv.org/abs/2411.02059>

- [4] V. Zhong, C. Xiong, i R. Socher, „Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. 2017. Dostępne na: <https://arxiv.org/abs/1709.00103>
- [5] J. Li, B. Hui, *i in.*, „Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs”. 2023. Dostępne na: <https://arxiv.org/abs/2305.03111>
- [6] Y. Shi i et al. Tang, „A Survey on Employing Large Language Models for Text-to-SQL Tasks”. 2024. Dostępne na: <https://arxiv.org/abs/2407.15186>
- [7] M. Pourreza i D. Rafiei, „DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction”. 2023. Dostępne na: <https://arxiv.org/abs/2304.11015>
- [8] L. Wang *i in.*, „Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. 2023. Dostępne na: <https://arxiv.org/abs/2305.04091>
- [9] S. Yao, J. Zhao, *i in.*, „ReAct: Synergizing Reasoning and Acting in Language Models”. 2022. Dostępne na: <https://arxiv.org/abs/2210.03629>
- [10] LangChain, „Build a SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langchain/sql-agent#run-your-agent-in-studio>
- [11] H. Touvron *i in.*, „Llama 3: Open Foundation and Instruction Models”. 2024. Dostępne na: <https://arxiv.org/abs/2302.13971>
- [12] K. Ociepa, Ł. Flis, R. Kinas, K. Wróbel, i A. Gwoździej, „Bielik v3 small: Technical report”, *arXiv preprint arXiv:2505.02550*, 2025.
- [13] LangChain, „Build a custom SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langgraph/sql-agent>